

Customer Retention AI

Enterprise Intelligence Platform

Comprehensive Project Report

Project Version	v4.0
Report Date	May 2026
ML Model	XGBoost 2.1
LLM	LLaMA 3.3 70B (Groq)
Backend	FastAPI 0.115
Training Dataset	KKBox Churn Challenge
Model Accuracy	97% AUC 0.972
Synthetic Customers	3,200

CONFIDENTIAL — Internal Use Only

1. Executive Summary

This document presents a comprehensive technical and architectural overview of the Customer Retention AI platform — a full-stack, enterprise-grade system designed to predict, explain, and act upon customer churn risk in real time. The platform integrates machine learning, explainable AI, large language models, and automated email campaigns into a single cohesive product.

97%	0.972	87.4%	82.5%	0.85
Model Accuracy	ROC-AUC Score	Churn Precision	Churn Recall	F1 Score

The system operates across three principal layers: a trained XGBoost classification model for fast churn scoring, a SHAP TreeExplainer layer for model interpretability, and a Groq-hosted LLaMA 3.3 70B large language model that generates structured, bilingual (English and Arabic) retention strategy reports. These layers are exposed through a FastAPI backend and consumed by a Vanilla JS enterprise dashboard.

2. Project Overview & Goals

The Customer Retention AI platform was built to address a fundamental business challenge: identifying customers who are likely to cancel their subscription before they actually do, and automatically generating intelligent, actionable retention strategies to win them back.

2.1 Core Business Objectives

- Reduce customer churn rate by scoring every customer for departure risk.
- Prioritise retention resources by routing high-risk accounts to human agents and low-risk accounts to automated workflows.
- Explain why a customer is at risk using SHAP feature attribution, providing transparent and auditable AI decisions.
- Generate personalised retention strategies in both English and Arabic using a state-of-the-art LLM.
- Automate email campaigns triggered by risk thresholds, with per-customer personalisation.

2.2 Key Capabilities Built

Capability	Description
Fast Churn Prediction	XGBoost inference returning a risk percentage in ~50ms
Deep AI Analysis	SHAP explainability + LLaMA bilingual report in ~3–5 seconds
Email Campaign Engine	Automated and bulk retention emails with tracking
Enterprise Dashboard	KPI cards, charts, customer table, risk heatmap, realtime alerts

Bilingual AI Output	All AI insights delivered in English and Arabic side by side
CSV Bulk Scoring	Batch upload for offline customer scoring
Next Best Action (NBA)	Ranked retention offers by effectiveness and cost
PDF/Print Reports	Per-customer printable reports from the dashboard

3. Dataset & Data Engineering

3.1 Source Data

The model was trained on the KKBox Churn Prediction Challenge dataset sourced from Kaggle. KKBox is a music streaming platform, and the dataset contains historical subscription and transaction records representing real-world customer billing behaviour.

Primary File	train_v2.csv (churn labels per user)
Transaction File	transactions_v2.csv (billing events)
Sampled Users	20,000 (stratified to preserve churn ratio)
Transaction Records	23,583 filtered for sampled users
Sampling Method	Stratified random sampling

3.2 Feature Engineering (RFM Analysis)

Seven engineered features were derived from raw transaction records using a Recency-Frequency-Monetary (RFM) framework adapted for subscription billing:

Feature	Description
total_transactions	Count of all payment transactions for the user
total_cancellations	Sum of explicit cancellation events
auto_renew_count	Number of transactions where auto-renewal was active
total_amount_paid	Cumulative lifetime revenue from the customer
avg_plan_price	Average subscription plan price across all billing periods
billing_tenure_days	Days elapsed between first and last transaction
cancel_rate	Cancellation rate = total_cancellations / total_transactions

3.3 Synthetic Customer Store (Runtime Demo Data)

In addition to model training data, the backend generates 3,200 synthetic customers at startup using a deterministic random generator. Each customer has a realistic risk score derived from a custom sigmoid scoring formula that incorporates cancel rate, auto-renew history, tenure, VIP

status, and segment. This allows the dashboard to demonstrate all analytical features without requiring a live database.

4. Machine Learning Pipeline

4.1 Algorithm & Configuration

The churn prediction model uses XGBoost (eXtreme Gradient Boosting), a decision-tree ensemble algorithm well-suited to tabular data with class imbalance. XGBoost was chosen for its speed, built-in regularisation, and native SHAP compatibility.

Algorithm	XGBoost (XGBClassifier)
Hyperparameter Search	RandomizedSearchCV
max_depth	6
learning_rate	0.05
scale_pos_weight	5 (handles class imbalance)
subsample	0.9
colsample_bytree	0.8
Early Stopping	Tree #28 of max 1,000
Model Format	JSON (ai_retention_xgboost_optimized.json)

4.2 Threshold Optimisation

The default XGBoost threshold of 0.5 was replaced with an optimal threshold of 0.633, determined by maximising the F1 score on a held-out validation set using precision_recall_curve. This raised churn precision from 86.2% to 87.4% while maintaining strong recall.

4.3 Model Performance

97% Accuracy	0.972 ROC-AUC	87.4% Precision (churn)	82.5% Recall (churn)	0.85 F1 (churn)	0.633 Optimal Threshold
------------------------	-------------------------	--------------------------------------	--------------------------------	---------------------------	--------------------------------------

4.4 Retention Action Engine

After prediction, a rule-based RetentionActionEngine maps each customer’s risk and VIP status to one of four business decisions:

Priority	Decision Code	Action Taken
CRITICAL + VIP	TRIGGER_LLAMA_AND_ALERT_HUMAN	Immediate human escalation + LLM analysis
CRITICAL / HIGH	HUMAN_REVIEW_AND_SAVE_OFFER	Human review with personalised save offer
MEDIUM	SEND_AUTOMATED_DISCOUNT	Automated discount email triggered
LOW	NO_ACTION_REQUIRED	Standard lifecycle journey continues

4.5 Priority Scoring Logic

Each customer receives a numeric priority score (0–100) computed as: $\text{risk} \times 0.76 + \text{VIP bonus (14 if VIP)} + \text{cancel_rate} \times 12$. This score drives the escalation tier:

- Score ≥ 85 , or VIP with score $\geq 70 \rightarrow$ CRITICAL
- Score $\geq 64 \rightarrow$ HIGH
- Score $\geq 40 \rightarrow$ MEDIUM
- Score $< 40 \rightarrow$ LOW

5. Explainability Layer — SHAP

SHAP (SHapley Additive exPlanations) provides per-prediction feature attribution, answering the question: “Which features contributed how much to this specific customer’s churn risk score?”

5.1 Implementation

- SHAP TreeExplainer is used, optimised for tree-based models like XGBoost.
- SHAP is imported conditionally (try/except) so the system degrades gracefully if the library is not installed.
- When SHAP is unavailable, the system falls back to a rule-based feature effect calculation using business logic.
- Per-customer SHAP values identify which of the seven features increased or decreased churn probability.

5.2 SHAP Output Structure

Each feature effect returned includes: the feature label (English + Arabic), the raw value, the SHAP impact as a percentage change, the direction (increases_churn vs reduces_churn), a relative strength score (0–100), and a human-readable explanation in both languages.

5.3 Top Churn Drivers (Rule-Based Fallback)

Feature	Direction	Business Interpretation
---------	-----------	-------------------------

Cancel Rate	increases_churn	High historical cancellation rate is the strongest signal of future departure.
Auto-Renew Count	reduces_churn	Customers who have repeatedly auto-renewed are less likely to churn.
Billing Tenure (days)	reduces_churn	Longer tenure indicates loyalty and reduces volatility.

6. LLM Integration — Groq LLaMA 3.3 70B

6.1 Model & Provider

The platform integrates Meta's LLaMA 3.3 70B Versatile model, accessed through the Groq API (api.groq.com). Groq is chosen for its ultra-low-latency LLM inference infrastructure, achieving 3–5 second end-to-end response times including SHAP computation.

Provider	Groq (api.groq.com)
Model	llama-3.3-70b-versatile
Output Format	Structured JSON (not raw markdown)
Languages	English + Arabic (bilingual by default)
Trigger	On-demand via /api/v1/analyze-risk-detailed
Configuration	.env file (LLAMA_API_KEY, LLAMA_API_URL, LLAMA_MODEL_NAME)

6.2 Structured JSON Output

The LLM is prompted to return structured JSON rather than free-form text. This enables the frontend to render insights as typed UI components (cards, timelines, panels) rather than raw paragraphs. The structured output contains the following fields for both English and Arabic:

- `churn_risk_summary` — one-sentence risk assessment
- `behavioral_diagnosis` — analysis of customer behaviour patterns
- `root_causes_ranked` — ordered list of churn drivers
- `recommended_rescue_strategy` — high-level retention plan
- `empathy_guidance` — tone and sensitivity instructions for agents
- `suggested_agent_script` — ready-to-use call script for retention agents
- `executive_takeaway` — one-line board-level summary

6.3 Context Passed to LLM

The LLM prompt includes: the customer's full feature set, the XGBoost churn risk percentage, the SHAP feature effects (ranked by impact), the customer's VIP status and priority tier, and explicit bilingual output instructions. This grounds the LLM response in quantitative model output rather than generic advice.

7. Backend Architecture — FastAPI

7.1 Application Structure

The backend is a single FastAPI application (main.py) serving both the REST API and the frontend static files. The application loads at startup: the XGBoost model, the customer store, and conditionally SHAP. Environment variables are parsed from a .env file.

7.2 API Endpoint Reference

Method	Endpoint	Description
POST	/api/v1/analyze-risk	Fast XGBoost prediction (~50ms)
POST	/api/v1/analyze-risk-detailed	SHAP + LLM deep analysis (~3–5s)
GET	/api/v1/customers	Paginated customer list (search, filter, sort)
GET	/api/v1/customer/{id}	Single customer with full analytics
GET	/api/v1/dashboard-overview	Dashboard KPI summary
GET	/api/v1/analytics	Charts: distribution, heatmap, trends
POST	/api/v1/customers/upload-csv	CSV bulk upload and scoring
DELETE	/api/v1/customer/{id}	Remove customer from store
GET	/api/v1/llm-analysis/{id}	Retrieve stored LLM analysis
GET	/api/v1/realtime	Live alerts feed
GET	/api/v1/health	System health check
POST	/api/v1/email/send	Send single retention email (async)
POST	/api/v1/email/send-sync	Send single retention email (sync)
POST	/api/v1/email/send-bulk	Send emails to multiple customers
POST	/api/v1/email/campaign	Trigger bulk campaign by risk filter
GET	/api/v1/email/status/{id}	Track email delivery status
GET	/api/v1/email/history/{cust_id}	Customer email history
GET	/api/v1/email/campaign-dashboard	Campaign summary and logs
GET	/api/v1/email/preview/{risk_level}	HTML preview of email template

7.3 Data Validation

All incoming requests are validated by Pydantic v2 models. The CustomerData model enforces type safety, non-negative numeric fields, and string length bounds on user_id. Invalid requests return HTTP 422 with structured error details automatically.

7.4 Fast vs Deep Analysis Separation

A deliberate architectural decision separates lightweight from expensive inference:

- /analyze-risk runs XGBoost only and returns in ~50ms. This is suitable for real-time scoring of every incoming customer.
- /analyze-risk-detailed additionally runs SHAP and calls the Groq LLM API. At 3–5 seconds, this is triggered on-demand for customers flagged for intervention.

This separation is critical for enterprise scalability: avoid expensive LLM calls at high request volume while still providing deep insights when human judgment is needed.

7.5 Persistence & Storage

Customer data is persisted to a JSON flat file (retention_customer_store.json). On startup, existing customers are loaded from disk into in-memory dictionaries (CUSTOMERS list and CUSTOMERS_BY_ID hash map) for O(1) lookup by ID. Any new customer analysed via the API is appended and saved. The architecture is designed for straightforward migration to PostgreSQL.

8. Email Campaign Engine

A dedicated email campaign subsystem was built to automate retention outreach directly from the API. It consists of three service modules: email_service.py, email_tracker.py, and email_templates.py.

8.1 Email Service

- Sends retention emails via Gmail SMTP over SSL (port 465).
- Supports single-send, bulk-send, and campaign-triggered modes.
- Uses FastAPI BackgroundTasks for non-blocking async delivery.
- Configurable via SENDER_EMAIL and SENDER_PASSWORD environment variables.

8.2 Email Templates

Risk-tiered HTML email templates are generated dynamically based on the customer's risk percentage. Three template tiers exist: Low Risk (0–39%), Medium Risk (40–69%), and High Risk (70–100%). Templates are fully HTML/CSS formatted and include personalised messaging. A preview endpoint (/api/v1/email/preview/{risk_level}) allows teams to inspect templates before a campaign.

8.3 Email Tracking

Every email sent receives a unique tracking ID. The tracker maintains a status log (QUEUED → SENDING → SENT / FAILED) and exposes per-email status lookups, per-customer email history, and a campaign dashboard summary. This enables post-campaign effectiveness measurement.

8.4 Campaign Modes

Mode	Endpoint	Behaviour
Single (async)	POST /email/send	Non-blocking; returns immediately with queued status
Single (sync)	POST /email/send-sync	Blocks until sent; full result returned
Bulk by ID	POST /email/send-bulk	Send to list of customer IDs
Campaign by risk	POST /email/campaign	Filter by all / low / medium / high risk band

9. Frontend Dashboard

The frontend is a Vanilla JS single-page application served as static files by FastAPI. No framework is required; the dashboard is built with HTML5, CSS3, Chart.js, and Lucide icons.

9.1 Dashboard Views

View	Contents
Overview	KPI cards, churn distribution pie chart, retention trend line, revenue at-risk bar chart, AI action distribution doughnut
All Customers	Searchable, sortable, filterable paginated table. Risk heatmap, segmentation chart. Click any row for a customer detail drawer.
AI Analysis	Single-customer scoring form, CSV bulk upload, structured AI insights panel with Advanced Analysis (SHAP + LLM) button.
Realtime	Live alerts feed, activity log, VIP vs. non-VIP risk comparison chart.

9.2 Frontend Features

- Dark / Light mode toggle with local persistence
- Customer detail drawer: full analytics, monthly risk trend chart, action history, LLM insights
- PDF / print report generation per customer and overview
- Bilingual display: English and Arabic shown side by side throughout
- Responsive layout: adapts to mobile, tablet, and desktop
- CSV bulk upload UI with progress feedback

9.3 Frontend ↔ Backend Communication

The frontend uses the browser Fetch API to call JSON endpoints. User form inputs are serialised to JSON matching the CustomerData Pydantic model. Fast prediction results are rendered immediately. The Advanced Analysis button fires a second call to the detailed endpoint, then re-renders the insight panel with SHAP effects and LLM narrative.

10. Technology Stack

Layer	Technology	Role
ML Model	XGBoost 2.1	Trained on KKBox dataset
Explainability	SHAP (TreeExplainer)	Per-prediction feature attribution
LLM	LLaMA 3.3 70B via Groq	Bilingual structured retention reports
Backend Framework	FastAPI 0.115	Async REST API + static file serving
Data Validation	Pydantic v2	Request/response type safety
ASGI Server	Uvicorn	Production-ready async server
Data Processing	pandas, numpy	Feature frame construction
HTTP Client	requests	Groq LLM API calls
Email	smtplib + MIME (Gmail)	Retention campaign delivery
Frontend	Vanilla JS (HTML5/CSS3)	Zero-dependency SPA
Charts	Chart.js	Interactive dashboard visualisations
Icons	Lucide	Consistent icon system
Python Version	3.12	Type hints throughout

11. End-to-End System Workflow

The following steps describe the complete request lifecycle from data entry to automated action:

1. Agent or system submits customer billing features (7 fields) via POST /api/v1/analyze-risk.
2. Pydantic validates the payload; missing or invalid fields return HTTP 422.
3. build_feature_frame() aligns features to the XGBoost model's expected column order.
4. XGBoost predict_proba() returns a churn probability; applied against threshold 0.633.
5. priority_from_score() assigns a risk tier (LOW / MEDIUM / HIGH / CRITICAL).
6. decision_from_priority() maps tier and VIP flag to a business action code.
7. build_structured_llm_analysis() generates bilingual rule-based insights as a fallback.
8. Result is saved to CUSTOMERS in memory and persisted to JSON store.

9. If Advanced Analysis is requested: SHAP computes per-feature effects; Groq LLaMA is called with SHAP context; structured JSON report is returned.
10. If risk is HIGH or CRITICAL: the email service queues a personalised retention email to the configured SMTP address.
11. Frontend renders risk score, priority badge, SHAP cards, LLM narrative, and action timeline.

12. Project History — What Has Been Done

Phase 1 — Core Analytics Platform (Complete)

- Enterprise dashboard with KPI cards and interactive Chart.js visualisations.
- Customer table with search, filter, sort, and pagination.
- AI prediction form with structured insights panel.
- Dark/light mode with responsive layout for mobile and desktop.
- 3,200 synthetic customer dataset generated deterministically.

Phase 2 — Advanced AI Layer (Complete)

- SHAP integration for real model-based feature explainability.
- Groq LLaMA 3.3 70B API integration with full prompt engineering.
- Structured bilingual JSON output replacing raw LLM text.
- Next Best Action (NBA) recommendation engine with ranked offers.
- Separate fast and deep analysis endpoints for enterprise scalability.
- Threshold optimisation using precision-recall analysis (0.633).

Phase 3 — Email Campaign Engine (Complete)

- Risk-tiered HTML email templates (low / medium / high).
- Single, bulk, and campaign-mode email delivery via Gmail SMTP.
- Email tracking system with status lifecycle and history per customer.
- Campaign dashboard endpoint with delivery statistics.
- Async background task delivery for non-blocking API responses.
- Template preview endpoint for QA before campaign launch.

Phase 4 — Operational Hardening (Complete)

- Fallback scoring logic if model file is not found (sigmoid formula).
- Graceful SHAP degradation when library is not installed.
- .env file parsing for secrets (no hardcoded credentials).
- Structured logging throughout the backend (timestamped, levelled).
- CORS middleware configured for cross-origin frontend access.

- JSON persistence layer for customer store with load-on-startup.

13. Future Improvements & Roadmap

13.1 Infrastructure & Data

- PostgreSQL integration: replace the JSON flat file with a relational database for concurrent write safety, query flexibility, and production scalability.
- Authentication & RBAC: add JWT-based authentication and role-based access control (Admin, Analyst, Agent) to the API and dashboard.
- WebSocket real-time alerts: push risk score changes and new high-risk customers to the dashboard in real time without polling.
- Redis caching: cache SHAP computation results and LLM outputs per customer to avoid redundant expensive calls on re-analysis.

13.2 Model Improvements

- Retrain on production data: once real customer data is available, retrain XGBoost to replace the KKBBox synthetic approximation with domain-specific patterns.
- Feature enrichment: add support ticket frequency, product usage depth, NPS score, and payment failure events as additional churn signals.
- Model versioning: implement MLflow or a similar registry to track model versions, compare performance across retraining runs, and enable rollback.
- Online learning: explore `partial_fit` approaches to update the model incrementally as new labelled churn events are observed.
- Champion/Challenger A/B testing: route a percentage of predictions to a challenger model to compare real-world business outcomes.

13.3 LLM & AI Enhancements

- LLM caching layer: cache LLM outputs by customer feature hash to reduce Groq API costs by reusing identical analyses.
- Prompt versioning: version-control LLM prompts and track which prompt version produced each analysis for auditing.
- Additional languages: extend bilingual support to French, Spanish, or other languages relevant to the customer base.
- LLM evaluation: implement automated quality scoring of LLM outputs using a secondary LLM judge to ensure retention strategy quality.
- Fine-tuned model: explore fine-tuning a smaller LLM on validated retention strategy examples to reduce latency and cost.

13.4 Email Campaign Improvements

- Email open and click tracking: integrate tracking pixels and link redirects to measure campaign engagement rates.

- A/B testing for email content: test multiple subject lines and offers against control groups, measure retention lift per variant.
- Unsubscribe management: add preference centre and suppression list to comply with CAN-SPAM and GDPR.
- Dynamic content personalisation: inject customer-specific SHAP insights directly into the email HTML for hyper-personalised messaging.
- Drip campaign scheduling: implement multi-step retention journeys with timed follow-up emails based on customer response.

13.5 Dashboard & UX

- PDF export with charts: render Chart.js charts server-side (e.g. via Puppeteer) and embed them into downloadable PDF reports.
- CSV bulk upload improvements: show per-row scoring progress, allow re-scoring of existing customers, and surface anomalies in uploaded data.
- Cohort analysis view: group customers by acquisition month and visualise churn rates across cohorts over time.
- Operator notes: allow retention agents to annotate customer records with call outcomes and save-offer responses.
- Mobile app: wrap the dashboard in a React Native or PWA shell for field agents who need on-the-go access.

14. Architecture Decisions & Rationale

Fast/Deep endpoint split	Separates ~50ms ML scoring from ~5s LLM calls. Enables high-throughput bulk scoring without incurring LLM API costs on every request.
Structured LLM output	JSON output instead of raw text allows the frontend to render typed UI components. Eliminates markdown-in-UI problems and supports field-level bilingual rendering.
Conditional SHAP import	SHAP is a heavyweight C++ dependency. Wrapping it in try/except means the system works in minimal environments and degrades gracefully.
In-memory + JSON store	Avoids database setup complexity during development. The service-layer pattern means switching to PostgreSQL requires only changing the load/save functions.
Bilingual by default	Arabic is a first-class output, not a translation layer. LLM prompts request parallel Arabic content, ensuring professional quality rather than machine translation.
VIP_PLAN_PRICE threshold	avg_plan_price > 500 triggers VIP classification, adjusting priority scores and action routing. This threshold is configurable via the constant.
Background email tasks	Email delivery uses FastAPI BackgroundTasks so the HTTP response returns immediately. This prevents SMTP latency from blocking the API

	caller.
--	---------

15. Cumulative Done-By History

The following table records all major deliverables completed across the project lifecycle, which will continue to be updated as new milestones are reached.

Phase	Deliverable	Detail
Phase 1	ML Training	XGBoost model trained on KKBox dataset with RandomizedSearchCV and threshold optimisation. ROC-AUC 0.972.
Phase 1	Feature Engineering	RFM-based 7-feature pipeline: transactions, cancellations, auto-renew, amounts, tenure, cancel rate.
Phase 1	FastAPI Backend	Core API with Pydantic validation, XGBoost inference, business rule engine, and JSON persistence.
Phase 1	Vanilla JS Dashboard	SPA with KPI cards, Chart.js charts, customer table, dark/light mode, responsive layout.
Phase 1	Synthetic Data	3,200 deterministic synthetic customers with realistic risk distributions and segment profiles.
Phase 2	SHAP Integration	TreeExplainer with fallback rule-based effects. Per-feature direction, strength, and bilingual explanations.
Phase 2	Groq LLM Integration	LLaMA 3.3 70B via Groq API with structured JSON prompting and bilingual output.
Phase 2	NBA Engine	Next Best Action ranking with ROI scoring, offer tiers, and cost estimates.
Phase 2	Endpoint Separation	Distinct /analyze-risk (fast) and /analyze-risk-detailed (SHAP+LLM) endpoints.
Phase 2	Bilingual Output	Full English + Arabic throughout: LLM reports, feature labels, timelines, action steps.
Phase 3	Email Service	Gmail SMTP email delivery with async background tasks and sync fallback.
Phase 3	Email Templates	Risk-tiered HTML templates (low/medium/high) with dynamic personalisation.
Phase 3	Email Tracker	UUID-based delivery tracking with status lifecycle, history, and campaign dashboard.
Phase 3	Campaign API	Four campaign modes: single async, single sync, bulk by ID, bulk by risk filter.
Phase 4	Model Fallback	Sigmoid-based fallback scoring when XGBoost model file is absent.
Phase 4	Env Config	.env parsing for LLAMA_API_KEY, SENDER_EMAIL, SENDER_PASSWORD — no hardcoded secrets.

Phase 4	Structured Logging	Timestamped, levelled logging throughout the backend for operational visibility.
Phase 4	CORS Middleware	CORSMiddleware configured for cross-origin frontend access.

This document will be updated on every future milestone to maintain a continuous project history.

End of Report — Customer Retention AI Enterprise Platform